

ДЗ 1

А - В REPL выполнить команды `type(3)`, `type(3.0)`, `type("3")`, `type(3 == 3)`, `type(None)` и. Записать результаты.

Решение:

```
elizaveta@MacBook-Air-Elizaveta ~ % python3
Python 3.9.6 (default, May 7 2023, 23:32:44)
[Clang 14.0.3 (clang-1403.0.22.14.1)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> type(3)
<class 'int'>
>>> type(3.0)
<class 'float'>
>>> type("3")
<class 'str'>
>>>
>>> type(3 == 3)
<class 'bool'>
>>> type(None)
<class 'NoneType'>
>>>
```

3 - целое число, int

3.0 - число с плавающей точкой, float

"3" - строка, str

3 == 3 - результат сравнения, bool (True)

None - специальное «пустое» значение, NoneType

Б - Показать, что $0.1 + 0.2$ не равно 0.3 . Сравнить сначала через `==` и через `math.isclose`.

Решение:

```
[>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
>>> import math
>>> math.isclose(0.1 + 0.2, 0.3)
True
>>>
```

Числа с плавающей точкой хранятся в памяти в двоичном виде и не могут точно представить 0.1 и 0.2, поэтому их сумма дает погрешность (0.30000000000000004).

Сравнение `==` требует абсолютного совпадения, поэтому оно ложно.

Функция `math.isclose()` сравнивает с учётом допустимой погрешности, поэтому возвращает True.

Вывод: для вещественных чисел почти никогда нельзя использовать `==`; нужно применять `math.isclose()`.

В - Программа: спросить два числа и операцию (+ - * // %), напечатать результат. Ловить нечисловой ввод.

Решение:

```
# Горохова Е.С.
# Задание В: калькулятор с двумя числами и операцией

def get_number(prompt):
    """Запрашивает число, пока пользователь не введёт корректное значение."""
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Ошибка: это не число. Попробуйте снова.")

a = get_number("Введите первое число: ")
b = get_number("Введите второе число: ")
op = input("Введите операцию (+, -, *, /, //, %): ").strip()

if op == "+":
    result = a + b
elif op == "-":
    result = a - b
elif op == "*":
    result = a * b
elif op == "/":
    if b == 0:
        print("Ошибка: деление на ноль")
        result = None
    else:
        result = a / b
elif op == "//":
    if b == 0:
        print("Ошибка: деление на ноль")
        result = None
    else:
        result = a // b
elif op == "%":
    if b == 0:
        print("Ошибка: деление на ноль")
        result = None
    else:
        result = a % b
else:
    print("Ошибка: неизвестная операция")
    result = None

if result is not None:
    print("Результат:", result)
```

try / except ValueError ловит нечисловой ввод, т. е. программа не падает, а просит ввести заново

Отдельно обрабатывается деление на ноль для /, // и %.

Неизвестная операция тоже даёт понятную ошибку.

Тестирование программы:

```
elizaveta@MacBook-Air-Elizaveta Documents % python3 praktic_1_calculator.py
Введите первое число: 1
Введите второе число: 3
Введите операцию (+, -, *, /, //, %): +
Результат: 4.0
```

Рисунок 1. Тест ввода чисел 1;3

```
elizaveta@MacBook-Air-Elizaveta Documents % python3 praktic_1_calculator.py
Введите первое число: 6
Ошибка: это не число. Попробуйте снова.
Введите первое число: █
```

Рисунок 2. Тест нечислового ввода

```
elizaveta@MacBook-Air-Elizaveta Documents % python3 praktic_1_calculator.py
Введите первое число: 7
Введите второе число: 0
Введите операцию (+, -, *, /, //, %): /
Ошибка: деление на ноль
```

Рисунок 3. Тест операции с нулем

```
elizaveta@MacBook-Air-Elizaveta Documents % python3 praktic_1_calculator.py
Введите первое число: 1
Введите второе число: 1
Введите операцию (+, -, *, /, //, %): x
Ошибка: неизвестная операция
```

Рисунок 4. Ввод неизвестной операции

Г - Объяснить своими словами, почему $a = 5$, $b = a$, $a = 7$ не меняет b . 5-7 предложений

Решение:

Когда пишем $a = 5$, Python создаёт в памяти число 5 и привязывает к нему имя a . То есть a это просто ярлык, который указывает на пятёрку.

Когда пишем $b = a$, Python не копирует саму пятёрку, а просто привязывает второе имя b к тому же самому числу 5. Теперь и a , и b указывают на одну и ту же пятёрку.

Если пишем $a = 7$, команда не меняет число 5, потому что числа в Python нельзя изменить, они неизменяемые. Вместо этого Python создаёт новое число 7 и переставляет ярлык a на него. А ярлык b так и остаётся висеть на старой пятёрке. Поэтому b по-прежнему равно 5.

То есть a и b это две разные стрелки, и когда мы переводим стрелку a на другой объект, стрелка b остаётся там, где была.